

ActInf ModelStream 018.1: Robust Decision-Making Via Free Energy Minimization (Russo & Jesawada)

Source: [YouTube](#) Playlist: ModelStreams Duration: 1:26:23 | Uploaded: Unknown
Transcript method: auto_caption

Hello, welcome everyone. It is April 28th, 2025 and we're in active model stream 18.1 on robust decisionmaking via free energy minimization. So, thank you to the authors who have joined today. looking forward to this presentation and walk through. So thanks again and to you all. Go for it. Okay. Thank you Daniel. Um and thank you for the invite for inviting us here. So we uh are let's say that's three of us today. Arash unfortunately was the other first author of the paper that made most of the mathematics who cannot be here today. But I will we will we will basically go through the uh the the the paper that we have just submitted. Uh so let me start uh with thanking all the co-authors. So these are the co-authors of uh the paper that is currently on the archive. Uh the paper provides a computational model uh which we call doctor free uh which relies basically on free energy minimization for policy computation and tries to install robustness into decision making. So our angle is to design policies making decisions that are robust against certain ambiguities. Uh our background so from the from the group in Salerash Jose and myself is that of control theory. So I apologize if sometimes maybe I would speak about some control terminology. So I will start with some motivation. Um first of all motivating the problem uh why uh we should be able to design agents autonomous agents that can make decision uh amid ambiguity. uh and we will uh let's say check some differences between what is probably uh natural intelligent how natural intelligence agents behave and autonomous agents current autonomous agents then I will jump into our solution for free energy minimization which is Dr. free this computational model I'm going to talk about and then I will let the floor to Josea for some discussion about the code to finally conclude with some remarks very well so um let's start to set out set out the ground first so um it is undeniable that autonomous agents have achieved groundbreaking performance uh for the example in the figure here in the slide on the left uh we can design today agents embodied in robots for example that can make very complex tasks like building towers from blocks as the the popular child game. Uh but if we look closely to the picture, the agent is performing this

task in a very controlled environment which is a lab and perhaps the policies that the agent learns. So the policies, the method that the agent uses to make actions are quite inflexible to changes against the environment and against the task. In contrast, natural intelligence, so for example, a child that is playing exactly the same game can do the same task in a much more robust way. And this is of course only a toy problem. Imagine what would happen if we had these mismatches between uh the with the environment between the training and the environment and what would happen if the agent starts to misbehave. We could have failures in the in the robot for example that could cause damages to the environment to the robot and maybe to people around them. So the angle that we are going to uh to discuss today is a is an approach based on the free energy minimization that tries to install robustness into the decision making problem. Again I speak here from the viewpoint of a a person that comes from control theory. Uh so we would like really to design policies so making decisions that allow agents to compute optimal actions that are also robust against certain mismatches that we will see soon. I know this is a much broader topic because there is a lot of research that could feed a lot of research programs here that spans from I don't know m sciences designing better actuators designing better sensors better software etc. Here we are really interested in the intelligent step and in fact this is a challenge that requires interdisciplinary foundations and the approach that we present is in fact interdisciplinary. It mixes a few key ingredients of course energy minimization uh that will serve us as a framework to cast policy computation as an optimal control problem. So with optimal control I mean the the discipline the problem of finding optimal actions that minimize a given cost function that formalizes the agent task and the way we will frame the problem we will see that this will have a lot to do with optimization in particular optimization in the space of densities in the probability space which is technically an infinite dimensional optimization problem. So the mix of three these three key ingredients gives us our computational model our free energy computational model which in fact consists of an infinite dimensional free energy formulation and a resolution engine that computes the optimal policy essentially and is able to output the optimal policy with provable guarantees. What is the gain uh of this approach? First of all, um it can provide a normative framework to equip uh natural agents to explain uh behaviors of natural agents. Then we could ever try to reverse engineer if we could distill this framework into a robot for example as we do in the experiments, we would be able to equip artificial agents with robust free energy minimization properties. And of course coming from a rather theoretical field like control it can provide some elegant mathematical methods. So the account that we use the unifying account that we

use for policy computation is free energy. Now free energy is very broad. Uh there is lots of literature. It really comes pops up everywhere in in several apparently different domains coming from of course neuroscience as a theory to explain self-organization brain functions and behaviors in general to technology tech some technological fields such as optimization mirror descent maximum entropy learn statistical learning etc. So this is really a unifying account that we are going to leverage for autonomous decision making. So let's start to formalize this decision making problem. So we are going to have an agent uh which will embody our model that is a free energy minimizing agent and the agent interacts with an environment. Now the the agent is equipped with a model of the world a world model which is obtained for example from training. So this train model we call it p_{θ} is basically any model of the world that the agent has available. Now it can come from training it can come from past experience it can come from any sort of different pieces of information. What is important is that this model is available offline to the agent and will ground and the agent will ground its decisions based on this model. Now the environment the world is as we know nonlinear stochastic and non stationary in general. So it is a challenge world the one we live in and it provides to the agent some information. This information we call it state x . This uh x is a label for both observations and beliefs that are relevant to the agent task. So x_{k-1} here is the information that is available to the agent in order to make a decision. They can be observations and can be belief functions. Based on this information, the agent well it has available the classical generative model and it has subject preferences that we express as cost functions and based on this uh as we will see today we will be computing a policy which is a randomized probability functions basically probability density function from which the agent computes an action U . Okay. So U_k is going to be this the set of actions that are determined by the agent and this these actions are obtained by sampling from the available information. Now what is the point? The point is that the environment the world is different by definition from the world model. Okay. So we would like to study what happens when the environment trained model differs from the real environment. Just in terms of notation as I said I'll try to keep the know the mathematics at the minimum. I just like to flag that random capital letters means random variables lowercase letters are just realizations and subscripts are time dependent. Whenever you see bold it is basically a vector. So we have this framework here. So we have this setup. We have an agent that is interacting with uh the world around it. We have essentially a feedback system where information in the forms of state feeds the agent. The agent computes an action and the actions pushes the world to move in a new state. So this is a

back and forth information that we can capture mathematically. Of course there is an initial condition. There is an initial state for the environment which is a prior p_{x_0} here. Then what happens? The agent given this initial information using the cost function the preferences and the generative model will determine an action in our framework. As I just illustrated, the action is obtained by sampling from a policy $P(u_k | x_k)$ given x_k minus one. This is the policy that the agent will use to generate u_k given the current information and this u_k once sampled is sent to the environment to the world and it will make it transition to a new state x_k . Now this is just a picture a snapshot at one time step this thing happens this thing happens continuously between zero and let's say some final time n and we can capture all of these as a product. So we have a product of these probability density functions or probability mass functions depending if we are working with continuous or discrete states. All in all, uh this big product of probability uh mass or density functions is just a joint a big joint probability density or a big joint probability mass function that in mat lab style annotation we indicate with p_n . So this closed loop probability function p_n captures the behavior the closed loop behavior of the agent. So it means that captures the interactions between the agent and the world. Now we could do technically exactly the same thing with the generative model. So we could make a big product of all the the conditional densities given some initial prior and we can call this probability uh function q_n . So this q_n is the joint probability density function of states and actions from the generative model. So the subject of our of our model, the goal of our model is that of computing the method to generate the action. So it's to compute the policies and the policies are going to be computed via free energy minimization. So how do we cast the free energy minimization problem? First of all, um we use the definition of variational free energy. Okay, so variation of free energy is essentially a complexity plus an expected cost and expected loss terms. Now complexity is statistical complexity. It is measured using the standard Kullback-Leibler divergence. So the Kullback-Leibler divergence I'm sure everybody that does active inference is familiar with. But it has basically this expression here. So it is not a distance because it is not symmetric but it is non-negative and it is zero if and only if p and q are the same. So we have this complexity term that quantifies how far the closed loop system p_n is from the generative model. So it is a sort of an error term and then there is an expected loss. Well, the expected loss is just the expectation over the closed loop behavior P_n of the summation between the state and the actual costs. The good news uh is that we know how to solve these uh control problems when basically the trained model is the same as the world model where basically you see in green here in the slide we have p_k being the trained model

and being exactly the same as the world model. So the good news is that we know how to solve this uh these optimization or optimal torque problems in a number of of very recent papers that we got that we got published uh and as a matter of fact we know how to compute policies in this setup. Let's consider one example that we will use throughout this live stream here which leverages a very nice platform from Georgia Tech which is called the robotium. The robotium is a robotic platform that allows you to deploy experiments remotely. So these guys from Georgia Tech basically provide you robots and provide you a small workspace where you can deploy your algorithms and they run and they run the experiments for you. So the robotarium uses uh a um a rover a unicycle robot where x_k the state is the position of the robot and u_k are the input speeds at the wheels. Now in the experiments that are also in the preprint that is attached to this live stream. Uh the robot has to move in an environment that is filled with obstacles. So whatever you see here in red are obstacles and the green position is the goal destination of the robot. So of course we would like to solve a navigation task where we reach the the green position while avoiding the reds. Now let's use let's minimize the free energy pretending that the world model and robot model are exactly the same. But let's assume let's say that we learn a model of the robot from a set of corrupted data. Let's say so that there is a very small difference between the real model of the robot which is the one in the bottom in blue and the the the the model obtained from training which is the one in red. So you see that the the difference is only in one parameter. Uh and the difference in that parameter is about 10%. So as an engineer speaking now 10% difference in in a parameter is not that much. So it's something that the system should be able to tolerate. Now let's consider let's set up the the policy computation framework. Let's say we have a very simple generative model uh where the the state distribution is a gaussian centered in the goal destination and Q the policy distribution a generative policy distribution is just a uniform distribution and let's say that the preferences the cost of the agent uh formalize collision avoidance with the obstacles. So our cost functions are basically something like the the one that you see in the bottom of the picture. So we have walls, mountains that emulate the presence of the obstacles and then you have a minimum in the bottom left area in the bottom left part of the work area minus one minus 1.5. Okay. So let's see what happens if we pretend actually if we don't know that the real robot is different from the robot that we learned during training. So let's apply the results that are known in the literature. So we just minimize this cost function on here in the in the policy space and what we see is uh something that is quite um uh quite relevant for us. What we see is that in this very simple setup by using the optimal policy and by only having a 10%

discrepancy between the real model and the model from training basically the robot fails in the task in basically every time except then in trivial situations by trivial situations I mean situations where the shortest path between the initial position of the robot and the final destination is obstacle free. You see that only in the red um uh in the in the red curve there is the only initial position that the robot is able to achieve the final destination. All the others whenever there is an obstacle along the path so that the robot should change direction to avoid obstacles the uh the the robot fails does not achieve the destination. Of course, this means if it was a real robot interacting in a real environment, this means that there is a crash with a wall or with the environment in general that can be even dangerous. This is exactly the type of behavior that we would like to avoid. So first of all, um this mismatch between the real robot and the robot model available during training is called ambiguity. So we have an ambiguous model and this ambiguity can have catastrophic effects. So it can lead to failures that can be catastrophic for the agent and for the surround surroundings. So in the next part of the presentation and then on will follow up with the code with implementations. We will discuss how our model can uh mitigate this uh this issue. So in particular our model relies on free energy minimization but we will see that basically we will minimize the worst case free energy and that means that implies a free energy maximization step which is quite unusual. Then the model will allow us to establish what are the best performance that can be achieved by an agent that follows the model plus will will have some interpretability um interpretability property. So the pol the pro the policy can be uh the cost driving the actions of an agent can be inferred from the viewpoint of an external observer and also clarifies some cognitive behaviors at least hinting hints tries to explain some cognitive behaviors in the in the in the regimes of large and small ambivities. So this is what the model will do. Now let's see what the model actually is and uh we should start by formalizing a little bit the problem and we should check why the robot was failing. We should understand why the robot was failing in the previous example when the ambiguity was not considered. So let's go back to our picture. In the end what happened before was that the real robot was different from the robot learned during training. So the two PDFs that were different and this small mismatch created the crashes that we saw in the picture just a few minutes before we can make a step an abstraction step and realize that we can look at these objects in the space of densities. Okay. Now these two objects are just two points in the space of densities. Let's say $\bar{P}$ is the training model, the nominal model that the agent has available and P is going to be a different point in this space. We are interested in protect protecting the decisions of the agent against all possible

perturbation in this space that are in a suitably suitably defined set. Okay. So what we are going to define is a neighborhood a sort of neighborhood although it's not properly mathematically a neighborhood which is centered around the nominal model and we are going to call this radius ϵ_a which we call radius of ambiguity. So the set inside the circle here is going to be called ambiguity set and it is going to be the radius of ambiguity. We can formalize things a little bit more. There are some technicalities that I refer to the the interested readers to the paper and we can say that basically this set this circle is the set of all probability densities that have Kullback-Leibler divergence from the nominal model of at most ϵ_a . Okay. Again it is an ambiguity radius quantifies how ambiguous the model the training model is can be state dependent time dependent and action dependent of course and it has a a a straightforward uh interpretation. Basically if it is small for a given action state pair this means that the agent is quite confident or believes it is confident in the model par in the training model. otherwise means that it it has low confidence in in the model that it is that it has available. So we have formalized this. Oh sorry there Carl maybe wanted to say something. I I saw a hand raising. You you did and I do want to say something. Um, sure. And I I I should apologize cuz I have to go in a few minutes, but I just wanted the opportunity to congratulate you on on on this work. Um before I go away uh you know there are I think a number of things that this work brings to the table and the first thing I think you've really highlighted is the intimate relationship between the sort of bio-inspired applications and the phenotypic principle and active inference and control theoretic um approaches which of course really start to matter when in real world implementations and neurobotics and the like. I I just wanted to acknowledge that's a really important point of convergence. Um and just also um foreground the key advance that you've framed and you're about I'm going to miss my favorite part because you're about to explain my favorite part but uh just to sort of put it in context. Um you know to my mind this brings a very fresh and different view of ambiguity to the table from the point of view of active inference because unlike um active inference in discrete states space based models where ambiguity is all about an intrinsic part of expected free energy ambiguity in this instance I think is much closer to the notion that you'd find in economics where it's not only uncertainty about what will happen if I do this, but it's I don't even know the game I'm playing. Um, and this is really literally a deep aspect of uncertainty. And basically, I think what you're bringing to the table here is a principled way of resolving that or incorporating that uncertainty, incorporating that uncertainty quantification into a vanilla free energy minimizing process. Um so to my mind you know this is very much and literally a deep solution um that I

you know one can look at in many ways. Someone can look at this as basically um a clever way of basian model averaging by including uncertainty about the very structure about the very um parameterization of the model itself by bringing hyper prize to the table that say look I you know I know what to do if this is the right model but I'm I'll have this extra kind of ambiguity this extra kind of uncertainty that actually inherits from not knowing what the model in and of itself is and you've provided a particular structure to that. I just want to conclude by um and this is something that you might pick up in later discussion. But it also um this this sort of kind of um robust basian model averaging or um ambig handling ambiguity not only accommodates the uncertainty about the very structure of the model itself but it also resolves something which is generic to variational approaches which is the overconfidence problem. Um so even if you knew the right model the mean field approximations that are inherent in variational bays and you know the minimization of variational free energy inevitably lead to an overconfidence uh and in principle I think the technology you're about to you know do a deep dive on will also resolve that overconfidence problem which has been longstanding um you know in in vari ational inference and of course really starts to bite when you actually put robots into play. So I'll stop there uh but just to thank you for involving me in this work and again congratulate you. Thank you K for the kind words and we can follow up on the many future and interesting directions that come from this work that start from this work. Okay. So I'm going to I'm going to continue. Uh so uh let me make a step back maybe. Uh so let me recall here the definition of ambiguity. Well the mathematical details here are not that relevant. Although the structure of this formula is important for our proofs. Whatever I'm going to show you is basically proof supported. So there is mathematics out there that supports our claimment claims. Of course here I'm going to keep it at a very high level. So we want to find this policy that is free energy minimizing but yet it is robust. So basically we want to find now a policy that find a method that finds the optimal policy minimizing the variation of free energy but across that ambiguity that Carl was also mentioning just a few seconds before. And how do we do that? So let me frame it first in English. Okay. So again we want to frame we want to find the optimal policy. So we are going to find a probability density function or a sequence of probability density functions. Uh P^* UK given X^K minus one. Now whenever you see the star symbol means optimality and the curly bracket over there means a sequence possibly this is a sequence of policies. So we want this solution this policy to be the solution of a problem where we minimize the variation of the energy. So the optimal policy is the arg is the minimizer of a cost functional that is complexity

plus expected cost just in the way we saw a few few a few minutes before but the ambiguity and we must take into account ambiguity. So we are not just minimizing this cost functional where we are actually accommodating for the worst case. So we are minimizing the maximum free energy where the maximum is taken across all possible environments all possible world models that belong to the ambiguity set that I just introduced. So we have a mean max problem where the minimization is as usual in the policy space but this minimization in the policy space happens on an inner maximization problem where we maximize over all possible words in the ambiguity set. This is the let's say uh the English translation of of of our of our framework. We can be a little bit more mathematical. Okay. So I'm just going to put the problem statement here which looks let's say a a huge equation but is not that complicated. So again we want to find the optimal policy the green the green part in the sentence and we want to minimize the variation of free energy which is nothing else than complexity and expected cost where complexity is again complexity between the closed loop system and the generative model and the expected cost is specified through the through the action and state cost and we want to minimize the worst case free energy across ambiguity which means that we want to minimize the maximum free energy across all possible environments that belong to the ambiguity set I just introduced. So this is basically the formulation the distributionally robust free energy minimization formulation that we have in the preprint that you see online that is also linked to the to the live stream. It is a challenging problem from the mathematical viewpoint because it is infinite dimensional and it is a mean max optimization problem. Um and it is also a nonlinear problem. So it is basically we cannot rely on linearity of the constraints or linearity of the cost function. Actually the the big step uh in the preprint online uh is that this problem can be solved and this basically gives us the resolution engine. The resolution engine is the set the software tools and mathematical methods that make it possible to solve that big optimization problem uh and make it possible to compute effectively a policy. So how does it work in really a nutshell? Um the agent which now we are exploring so we are dealing a little bit more details gets x_k minus one the state the current state beliefs or observations uses the generative model and the state cost. It will compute a policy and the policy will be computed by minimizing in a decursive way the variation of free energy the max variation of free energy. So what is the resolution engine? What will it do? And then will jump in the code. First of all it can break down at a temporal scale and at a logical level the original free energy problem. So there are lemas out there technicalities that tell you that the problem can be solved at different time steps and that minimization and maximization have a

precise order in which they should be performed. So first we should do maximization and then we should do minimization. So this gives you basically a bi level approach and these um software and lemmas uh what they do they yield you an explicit expression for the optimal policy. So we know what the optimal policy is going to be and also it tell you what is the lowest energy that the agent can achieve under ambiguity. So the optimal value from the optimal policy which is going to be a probability density function like the one that you see in the picture. We are going to sample an action with standard methods. So we are going to sample U_k send it to the environment and the loop will continue on and on. So let me start giving you some just the flavor of these key elements here. First of all, the policy can be computed in recursive way. In particular, it can be shown I don't do the details here that the policy can be computed as straight away as a minimization problem where the original problem is this is a a perturbation of the original problem where it will be apparent that to you that rather than the cost we had before now we have what we call the cost of ambiguity. We call this cost of ambiguity because it comes from an optimization problem. The the the the maximization step in the in the environment space that I mentioned before. But this cost is there just because there is ambiguity. If there was no ambiguity, we would not have this step and then we will recover classic methods, ambiguity free methods. So just two key takeaways here that there is a free energy maximization step. If you look at the top diagram top equation top formula in green you will realize that this is maximizing a variational free energy. So we have a free energy maximization step and it turns out that this free energy maximization step that gives us a cost of ambiguity is the guy that guarantees robustness. Also of course as also Carl was mentioning the model does not need the agent to know what is the real environment of the of the of in which the robot is moving. For example, the real robot model the real real robot model. um it only relies on the probability uh sorry on the on the ambiguity set B ea also in the top formula there is a C bar that this comes from a a backward recursion so one could use dynamic programming or neural approximators deep neural networks for example to estimate these pieces of this piece I'm not going into the details of this right So we have two steps as we said a breakdown a logical breakdown of the of the decision problem. So we want to find the policy. Finding the policy means minimizing that cost that is now at the center of the slide in the middle of the slide which has the cost of ambiguity and this cost of ambiguity comes from maximizing the free energy in order to make the model useful. So deployable essentially we need to find a way to compute the cost that cost of ambiguity cost of ambiguity that I remind you comes from this optimization problem now this is a very hard

optimization problem because it is infinite dimensional so this is optimization in the space of that bit okay and it is also a nonlinear problem although in a constraint that is a compact set it turns out that most of our methods the formal methods that we developed were aimed at proving that you can actually compute this cost in a rather uh efficient way at or at least with uh offtheshelf tools from convex optimization. Okay, I this is actually a set of theorems propositions and lema which I'm going to package as a meta theorem which is very informally stated. This meta theorem says that the cost of ambiguity uh has a specific form. It's EA plus C tilda. Okay. Well, EA is a given. EA is the radius of ambiguity and C is what our model computes. Now this C tilda comes from solving an optimization problem that now is a scalar optimization problem. So in this formula α is a non-negative scalar value. So we just have scalar optimization and bet the α is a function that I'm not going to specify here but it is convex. So in the end we can recast the problem of solving of finding the cost of ambiguity and address this problem via a convex scalar optimization problem that moreover has a global minimum of course because it is convex optimization. So we can go back to our resolution engine and say that that general step about uh finding the maximum free energy can be addressed with a scalar convex optimization problem. So we did not build a solver for this. We used offtheshelf tool. So that the problem that we are going to minimize now the cost functional that we are going to minimize has a very specific structure. You see that the cost of ambiguity is EA plus c . So the only thing that is needed to be to be done is to minimize this guy in the space of policies. Now this might oh sorry I forgot. So what what is the the the the engine actually doing? Taking the generative model it is computing a cost of ambiguity which many sometimes it has this double shaped potential here. So this is a cost of ambiguity means that in minus one and one the ambiguity is very low or or it is lower than other parts. Uh and for example towards the end of this diagram towards the size of this diagram the ambiguity is much larger. So the cost is much much larger. So what does our model do computes this cost of ambiguity and will output the policy and the policy comes from minimizing this cost functional here. Now this might seem as a very abstract um as a rather abstract um problem optimization problem but actually it is very popular in the literature and it even has a policy that has a well-defined algebraic solution which is a softmax. So what you see here is basically a softmax function a twisted softmax function and what what what is the idea uh that this formula that this policy implements? Well, the the the optimal policy expression is telling us that the optimal solution is given by the generative model Q . So the the bias of the agent, the the the beliefs of the agent in a sense and this model is somehow weighted and or distorted

by an exponential. Now the exponential um and what I think it is quite interesting is that the exponential assigns lower values that are associated to high ambiguity. So in the end the probability of getting an action that is very ambiguous is going to be lower and lower according to our model. Moreover, as I said before, we can also quantify what is the smallest possible free energy which has a log sum x expression which is standard in the literature from physics for example. So there is quite a lot on this slide. So I'd like to emphasize again the optimal policy the structure the optimal policy is nothing else than a twisted or distorted version of the generative model $Q(UK) \text{ given } XK \text{ minus one}$ and distortion happens to an exponential function and the exponential is lower when the ambiguity is large. So this means that the probability of sampling an action that is that is associated to high ambiguity is very small. So the policy in our model has a very clear and neat structure at least the algebraic structure. What happens inside the model? So we can continue developing the flow of information inside the model. So we have the cost of ambiguity computed with that scalar optimization problem. We gather the generative model and we build the exponential inside the policy formula. Then the two terms here are multiplied with each other and after normalization we obtain the policy. You see it is quite clear that the policy the optimal policy coming from multiplication and normalization is a compromise between the generative model and the ambiguity. So let me close the loop going back to the uh original problem that I was considering where the classical method would fail in this setting. So we have our rover unicycle robot that is moving in this environment filled with obstacles. We have the same generative model as before, the same training and robot model as before and the same cost. So exactly the same frame. And I remind you that when we don't take into account ambiguity, bad things can happen in the sense that the robot can crash in the obstacles unless this is really unless we really have a trivial situation. With our model instead with Dr. 3 starting exactly from the same initial conditions and starting with exactly the same setup, you see that the robot is able to be to navigate towards the destination. So for example, take the green the green trajectory the green path of the robot in the right side of the slide on the right side of the slide it will crash in the obstacle in the bottom obstacle. Instead in the left with our model in the left part of the slide with our model it is able to reroute in order to achieve the destination and reach the the final destination. Now uh I think I gave uh quite a few details uh and um probably the best thing is to jump to the code to see it in action unless there are questions or or comments that you would like to discuss at this stage. Looks awesome. Please continue then. Please I like to share the screen. Okay. Uh please confirm. Can you see my screen? Looks

good. Thank you. Okay. Uh hello everyone. My name is Josepha and I'm a postoc with Javanni at University of Salerno and my background is in control theory. Uh I would like to thank Daniel for inviting us for this presentation and I will present the uh code walk through for the robust decision making free energy minimization. So uh we will start with uh we start with the out uh first we'll start with the resolution engine uh then we'll discuss the robot experiments in more detail. What is the cost structure for this uh uh implementation? How we train the dynamics model? How we define the cost function? Uh what is the ambiguity radius and what is the cost of ambiguity? How is computed and at the end how we compute the policy basically and then we'll talk about the incilico and experimental results and then I will talk about in the end about the belief of that part which is a very cool part for of this Dr. free algorithm. Okay. So the resolution engine as uh Joanni just presented a few minutes ago uh we uh uh we want to solve a problem in which uh we want to obtain optimal policy a set of optimal policies uh which minimizes the maximum uh free energy uh cost basically and we have the uh we have our model the environment environments model the real the real environments model is inside a ball centered around a nominal uh model $\bar{P}$ and in the code I call $\bar{P}$ as a nominal model and uh so we we decompose this problem into a B level problem where we want to minimize over the free energy and the the maximum maximization of the free energy is implicitly incorporated in the terms of EA and CLI that which which makes the cost of ambiguity and we obtain by solving this problem we obtain the twisting kernel policy where uh EA and $\tilde{C}$ are the exponent of your exponential policy. Um so this is the flow of the Dr. free resolution engine. Now, so first I will introduce the robot experiments that uh we want we use the robotium platform by Georgia Tech. Uh and you can send your code to them and they will implement your code on the robots and you can they they will send you the video and the results basically. And what this is a real real snapshot of the platform of the work area and the robot a unicycle robot basically what what we want what our task is that uh we want to drive the robot successfully to the cross uh while avoiding the red obstacles. Basically uh the work area is around uh 3 cross 2 m in size and uh there are multiple robots if if you have multi- agent problem also it's a it's a good platform for for but for now we only consider a single agent here for simplicity. Um so this is what the code looks like. uh um so first we we define the global variables and action space um so so we import all the necessary libraries here the RPS is the robotarium python simulator uh it's a package by the robotarium platform uh and we call the robotarium which initializes a robot the transformation from the unicycle dynamics to the to the linear dynamics and all the Other

utilities we call `um` to solve the optimization problem for obtaining the cost of ambiguity we use `scipy` and it's a minimize tool to minimize the function and we use some other plotting functions like `math`, `math plot lib` and `skarn` to initialize our question process model. Um here I define the action spaces. So action spaces is basically the velocities of the robot in both axis x axis and y axis and we define it as a grid of a 5×5 discretization and it ranges from -0.5 to 0.5 the speeds of the robots in both directions. Um now after doing this first what we have to do is to load the dynamics model or train the dynamics model. So the $\bar{p}$ is the model that we have from the of the environment learn by some set of data. Um and here in our case we have three different data sets. X_1 , X_2 and X_3 are like the states of the robot at the positions and the U the U is the input or the velocity is given to that robot at particular time and we create a stack of these two to make the training input and y is our target. So the next possible state given the previous state and the current input what will be the next state is represented by y . So we have such three data sets each consisting of 500 samples and we train three different gaussian process models here and the so the gaussian process basically the main the most important component of a gaussian process model is the kernel function. So the kernel function is basically finding the distance between the two data points at a given time. So here the most widely used kernel function in a gaussian process modeling is a RBF kernel with some constant term added multiplied to it. So here we define the kernel function the constant term the RBF the radial basis function has a parameters called length scale and since we have a fourdimensional input we have four dimensional length scale parameter which is this is length scale is hyperparameter I'm just initializing it here plus some white noise because white noise because your data collection channel can be noisy And so to model that noise, we have a white kernel function as well here in the entire kernel function. Um now given this kernel function, we initialize the regression process regressor and we take this regressor and we start the fitting and we use the `fit` command basically for this and we obtain. So this is the kernel parameters and what we want to what after training what we obtain is the hyperparameters of these kernels. So the σ^2 here is the constant kernel coefficient the rbf here the radial basis function here has a parameter w or the length scale we have that length scale here and the noise term here. So these are the results of the three different data sets for the three different gaussian models. Now we take these models and then we move on to defining our cost function for the expected loss here. So the loss function is basically consisting of three

terms. So the first term in the cost is a squared distance between the goal point and the state. Uh the second term the cost sum actually is a RBF kernel again centered around the obstacles. So here uh are the location of the obstacles on the work area of the robotarium and we we calculate the RBF kernels. we center the RBF kernel around these blue points and uh we get the go sum. So basically what it's doing is that when you are near an obstacle the go sum the second term will have a very high value so it will shoot up uh and the third term which with a coefficient five here is for the walls. So we don't want our robot to go outside the outside the work area. So we don't want to get it outside the this rectangle. So basically it's like a it's like a control barrier function when uh if it's near the wall uh this fun this term has a very sharp rise uh and the cost is very high. So this is this uh this summarizes our cost function. And so the cost function takes uh the the initial state uh as an input and the goal points where you want to go the robot wants to go and the obstacle points uh as as an input and gives out a uh a scalar cost. So this is what uh when we compute this state cost this function for the entire grid this is what our function looks like. So you can see that you have a very high value when you're far from the goal and you have very high value near the obstacles and around the around the boundaries and you have the lowest value near the bottom left uh where the goal is in our scenario. Uh so we take this uh cost function and the and the gaussian process model and uh um now we come to a very important part of the resolution engine is the cost of ambiguity. So the cost of ambiguity as discussed before depends on the two uh two terms which is EA and cilda. The EA is the ambiguity radius and the CTa is the uh is a cost transformed cost uh which uh is the we define like this. So the cinda function takes an array of costs uh from the cost function. EA the ambiguity radius. Uh here the $\bar{P}$ is the nominal model probability function and the generative I I call the generative model the Q in the paper as a reference problem and it gives out the scalar again uh in the cost in the form of transform cost. So uh before Joani introduced a a term in the $\tilde{C}$ computation V_α uh this objective function is the V_α . So this is the objective function that we want to minimize basically and it depends on the in the parameter α and it consists of two terms. First is a fine term in α . So α into the ambiguity raises ea and the term two basically is α into the log some exponential uh of the of the cost with raised to 1 by α . So we want to minimize this objective function over α . Uh we take the minimize the minimum value of this objective function and we we basically compare it against the baseline which is computed by the maximization of this uh $\log x$ of the cost and the minimum value gives you $\tilde{c}$. uh for for more more detail I uh you can refer to the to the paper uh how it is formalized there. So this minimization gives you the

$\tilde{c}$ which is dependent on EA and α . So EA here we define EA as a scale divergence between the goal points and the next possible state computed from the caution process model and for computational stability we clip it between 0 and 100. Uh so K level is basically the difference between the two different distributions P and Q and oh here I made a mistake I think there should be Q here instead of P. So this is the expression for K divergence and for the gaussian distributions you can compute the scale divergence in an analytical form in a closed form without any sampling. So now uh given EA and $\tilde{c}$ we can compute our policy which are EA and $\tilde{c}$ makes the exponent of our policy. So the policy here the P^* the optimal policy is a twisting kernel where uh the q is the generative models policy and the exponential in exponential is uh the gives you high value if the $\tilde{c}$ the ambiguity cost is low. uh in this case particular case in this example we consider q to be uniform and since it is uniform you can basically uh remove it from this expression of the policy. So the policy computation code uh starts uh the policy computation code uh takes the initial state uh your control space because we have two inputs. You have two spaces the control or action spaces uh the world points and the obstacle points and gives out a uh in this case it will give out a 5×5 matrix of a probability uh uh it's a probability function from which you can also sample the action that you that that will go to the agent. Um so for this computation first we initialize the exponent inside the uh with the control space according to the control space and we also initialize the the policy uh distribution according to control space size. And now we we go we compute using the for loops for each possible combination of action. We take the combination of that action we give this action to the gaussian process model and we compute uh we predict the next state the next possible state and uh the gaussian process model will give you basically a mean and a covariance. So the here the next nominal is the mean of the gaussian process and the sigma norm is the is the covariance and using this uh we compute the samples the we sample from this uh mean and covariance uh the next possible states and using this sample um we we compute the we compute the cost uh sample the costs for this number of samples. So here okay. So now after after computing the samples I compute the EA the ambiguity radius again the ambiguity radius uh as mentioned before is a K divergence and it is clipped between 0 and 100 and uh I take the number of samples 100 number of samples and I compute the cost for uh all the next possible states uh and I give this cost matrix and the EA to the CTLA function and as uh I showed earlier uh it will give out give out another scalar value the transformed cost which is our this is our exponent the minus

the minus tilda is our exponent and uh so here we use a exponential trick to to normalize this uh and formulate the action PDF because uh to avoid any overflow and underflow we subtract the exponent with the maximum of that exponent matrix And basically you have the you have you have the normalized uh PDF here and we we do the flat flattening operation and unraveling the index and then we take the sample the action from that uh from this PDF and we give the action the the the agent will then give the action to the state uh to the system. So uh given this uh uh policy uh code now we we want to apply to the robotarium uh example and we perform uh basically uh 12 set of experiments on three different training stages. So the doc you can see in the top panel on the top is the doctor free algorithm and the on the bottom is the aware uh panel uh where you can see that the the doctor free agent even with unknown uh unknown system dynamics it can still reach the goal without uh uh colliding with any obstacles while the ambiguity unaware agent although the policy it is computing is also So um optimal it uh it encounters or collides inside the obstacle only in some cases when you can see that the obstacles are not present in the way of the agent it will reach the goal. So uh this is we we show that uh how our method uh basically solves the issue of uh uh the the model mismatch and we also have the experimental results for this. So you can see the agent uh the robot moving in the environment the actual robot and how it uh avoids the obstacle. So because of some uncertainty sometimes the robot also uh gets a bit confused let's say but but uh anyway it will it will reach the the destination without uh without colliding which is uh quite cool. Okay. Yeah, it's uh it gets Okay. Could you just share a little bit? Could you just share a little bit right there? What what is its spatial awareness like? Is it aware of its coordinates? Is it have a egocentric or alocentric navigation? Like how does it understand the location and the relative position of the obstacles? So, so, so the so yeah. Okay. Uh so uh here in this case um so the the the location is provided by the the camera. So the camera is tracking the top camera is tracking the robot and it it updates his location. The camera computes his location and updates the location on the on the grid. Also it it uh here it uh it has the input. It it knows the obstacles location. So it is pre pre is given in the cost function we define the obstacle points. So it knows the obstacles are are present. So when it comes near the obstacle the cost function is automatically very high and it's it avoids immediately the obstacle. I hope uh it answers your question. Okay. So it it's aware of its two-dimensional position and of the obstacle and the goals and it computes kind of like a suitability that's ambiguity aware. Yeah. Yeah. Yeah. Awesome. Thank you. But uh but it's but the but the thing it does not know is if it takes this action. So if it takes let's say from the grid uh from the action space it takes uh one

action. It does not know it. It it it does not know where it will take this action will take it. So it cannot foresee accurately where this action will take it. So that's where the ambiguity parts come. So that's where the ambiguity is. So it does not know exactly cannot be sure of its own decision. I I'll just ask one more. Oh yeah, please go ahead. Yeah. Okay. No, no, please ask. Yeah, I'll ask one more question from the live chat. So Andrew Pasha wrote, "I'm wondering about what degree of computational overhead this computing the worst case ambiguity adds to running the agents." So like just in this situation, what was kind of the computational complexity or the resource use of these different stages of the algorithm? like does the Dr. Free add a small overhead to uh most of the computational cost being on the policy inference or does this ambiguity add a significant computational overhead? Uh no uh it's uh for sure because of the com uh because of the ambiguity uh computation of the ambiguity uh cost there is a there it is a bit slower than the the ambiguity another agent but as you can see uh here in the sorry in the video uh it is all real time so it is doing the it is calculating the actions uh in real time uh I don't know the video is gone for some reason. Okay. Ah yeah if I if I may add something uh Daniel I think this is an excellent question because it may be a a a research research program per se. So finding the computational tricks that uh improve uh improve computational efficiency as much as possible. Yeah. So there are many tricks indeed. Yeah. So the one thing that uh would be interesting to see is because I can if if you allow me to go back a little bit. Okay. So policy computation here you can see that we are looping over all the possible actions here. But you don't need to do that. You can have a multi-threading or some parallel computation and you can compute all the actions for sorry the computation the cost for all the possible actions in a parallel computation and which will which will uh which will uh I think make it much more faster because here we are using the for loop over uh the it's uh so it's it has a if the if the action space is very large then it will for sure have a very uh high computational overhead. Right. But you can uh I think you can you can resolve it by parallel computation in the in for these four loops. Okay. If uh there are no more questions uh I will go ahead. Okay. So uh now we'll come to the belief update part. So belief update what is belief update is that um that uh if agent is performing some task we can learn the objective functions of the agent of the expert agent just by their data of the input and the states data of that agent and uh uh the cool part for the doctor free policy computation is that the policy is very interpretable. So you can reconstruct the objective function of the agent uh simply by using the data the state and the input data of the expert agent and we do this by by solving this uh by minimizing the likelihood negative likelihood uh given here. So the the likelihood function is

basically a summation of the aine term plus a log sum x and because both of the both of the terms the aine term and the log sum x are convex this optimization problem is convex even though the the cost function uh or the objective function uh of the expert agent is non-convex in nature. So the major the major components of for this uh belief update are the features the cost features and the weights the feature weights. The feature weights is the thing that we want to learn by optimizing over the the lock likel function and uh so first uh let's see the code structure for this to solve this problem. So again we define the global variables the action space we have to load the the train models uh for and the expert agents data. So we have the data of the state and the action for from some expert. Uh we define the cost features. Uh we compute the feature expectations uh using the gshian process model and we formulate and solve this optimization problem here and we obtain the recon reconstructed cost. So, so I will start with the cost features here. What what is the cost features that we define? So, similar to the previous uh task, I am defining the cost feature as the as the three as the two terms. The first is the the square distance from the goal point and the second term is again the gshian or the RBF kernel centered around some points and the RBF kernel is defined uh by this function here. Um so the RBF kernel now is not defined only on the obstacles the the actual obstacle in the environment but now the the RBF kernel is defined on a set of uniformly distributed points on the grid which may or may not represent the actual obstacles. So you can see that this is the the the feature the feature map or the feature uh grid and this is uniformly distributed and the actual uh actual obstacles are here and here. So we the so the algorithm has to learn how to how to give importance to the relevant features only and not to random features in the in the environment. Okay. So given this cost features uh we solve this minimization problem and u so we have let's say we have m number of samples from the expert agent. So we initialize the initial state and the uh and the the features. So for again for all the possible number of inputs we compute the next possible state using the uh gshian process model. And we sample a number of samples from from this question process model and we compute the features and uh we compute these features uh and we basically do the averaging or the uh we do the we do the averaging and we compute the expected value for these features. So here the first the second term uh of the uh of the equation was lock some exponential and for that you need to compute the features for all the possible inputs while the first term of the of the likelihood function was an aine term which only computes the given action taken from the expert agent. So you can see here at the end there are two terms. The first is the fine term in the W uh which which takes the feature sample which is dependent only on the input given from the agent. uh while

the log sum exponential uh is depending on all the possible inputs uh the feature computation for all the possible inputs and taking these two terms we stack for the entire uh time horizon m capital m and then we minimize it for the summation of this entire trajectory time trajectory. uh since it is a convex problem um you will have an optimal solution for any initialization of the weights. So for this pro for this problem particular problem uh we get this set of weights and you can see that the where there was obstacles in the grid in the in the center you have very high weights while all the other weights are near zero or are very low compared to the the real obstacles. So, so this was the real cost function or objective function of the expert agent and this is the reconstructed cost function of the obstacle of the of the uh agent and you can see that they are very similar in nature. So you have high cost away from the from the goal point and you have uh the obstacle in the center uh so on so forth and using this cost function the the reconstructed cost function the agent is still able to go to the goal while avoiding the obstacles. So just to recap, so the core structure for the robot routing task was to have the define the global variables, action space, load and train the dynamics model cost function and the features, compute the policy uh and you compute the the collect the data and basically visualize uh the robot uh uh going to the goal. While in the bic of that code structure, we also define the glo global. We define the cost features. Hello. Yeah, go ahead. It just blipped for a second. You're back though. Yeah. Okay. Yeah. Okay. Sorry. Uh so, okay. Um as I was saying so the belief update structure uh we also need to define global variables and action space. Uh we need to load the trained models. We define the cost features um and uh we compute the feature expectations and we formulate and solve the uh minimization of the log likelihood function and we obtain the reconstructed cost. So yeah. Okay. Uh okay. If you have any questions uh now I'm happy to take them. Awesome. Yeah, I'll ask a few uh quick questions and if anyone else has a question in the live chat, go for it. So, first just uh it was mentioned that it's infinite dimensional. So, could you unpack what that means as you're dealing with a finite number of dimensions in terms of sensors and actuators? So, what does that mean for it to be infinite dimensional? Okay. Ah the the infinite dimensional was actually here. So uh it's regarding to the maybe professor slide is better for this. I can uh maybe maybe I can put my slides because this point is more clear. Okay. Give me just one split second please. So to answer your question Daniel uh so do you think that infinite dimensional nature comes from the from the statement of the problem? So the so you are not you are not sharing I I I see it actually I see it. Ah okay. So maybe I'm not sure it's like on on Zoom like select maybe you're both sharing or something. Oh okay. But it's good though. I I see your I see your slides. You

see it. Okay. Perfect. Okay, the infinite dimensional that that was mentioned before is in the mathematics in the formal aspect of the uh of the framework because this optimization problem is uh has as decision variables the functions. Okay. And that makes it an infinity dimensional optimization. U when engineered uh Dr. Free on the code of course you need to estimate this function. So he made a step that discretizes at some point some of these functions uh and that of course reduces the problem to the uh finite dimensional optimization. So the infinite dimensional uh feature that uh we were referring to was about the the problem statement. So the fact that we are optimizing in functional spaces. Awesome. Okay. Another question like how would someone go about adapting this to another setting? Like how would they think about framing the kinds of uh algorithms and approaches that are already in the active inference or the cybernetics literature? Where where do you see opportunities like which systems for applying this? And then how adaptable or how would you go about adapting the open- source code you've provided to give this sort of ambiguity aware overlay? So I'll go maybe with the framework and then I'll let Josea comment more on the code since he developed the code. Um so uh well the framework is uh I would say rather general. Um so the I would say that the system level steps would be to identify a good cost functional uh identify the ambiguity set model the environment uh the training model and and everything else that comes with it. Once you have this key and and the generative model of course once you have these key ingredients I see um this framework fitting in the context of sequential policy optimization which is really sits is the bread and butter of control theory but also cybernetics and it has a lot of interdisciplinary links with with signal processing etc. In terms of code, I think I will let those have a answer that. But the main challenge is going to be the computation of the integrals and the uh uh pre probability density functions that are implied by the uh by the framework. Okay. Yeah. Uh so interesting question. Uh so there are uh I think robotics is one of the fields in sequential decision making which is very very cool to apply this because we have seen that when you have a mismatch in parameters of the models of the system uh the robotics uh the the task is very difficult to perform. We also also seen robots fail when they mismatch in the in the parameters. Um so if somebody wants to apply or implement this framework uh uh I would think that the they would first need to change the the the dynamics model that they have. So you need to adapt your dynamics model according to the system that you have first first of all. Um then uh for example if uh if you are in working in environment in which uh the how can I say the nominal model and the generative model are not really gaussian that at that point you will not have the analytical solution or the close form solutions as

given in the code and for that I think you need to revert to sampling some form of sampling to compute the integrals and the expectations. So that's one part that uh I think if uh if someone wants to implement this on a in a system which is more complicated and doesn't have gaussian assumptions then yeah you need to you need to consider sampling. Uh another part would be as I said if the action space of the of the of the system is too large then to avoid operational overhead you need to consider parallel computing maybe or some form of vectorization to to avoid the for loops in the input. Um yeah that's that's that's I think that's the that's the core path but other things like the cost of ambiguity the function that the function that computes this cost of ambiguity the structure would be the same for that. Yeah. Awesome. So, sort of in closing, what are your next steps for research or how do you see this continuing or what ways would you like to see people in the community kind of continue with the work that you're doing? Well, first of all, I'd say that if you want to use your code and you have any questions, I think we would be happy to to answer these questions and we are happy if the code is useful beyond of course the use case we are considering. Robotics being one one remarkable example I think and where do we see this placed? I think we prepared this slide that summarizes and maybe captures on some of the points that were discussed and about the interdisciplinary link. Uh so we see this uh placed in the context of feedback systems. So in the end the agent and the environment are two systems that are in feedback. So this the actions affect state and state affect the actions. Um and uh sorry and basically what we did was to bring some control theoretical and optimization methods into this minimization of variational free energy. So it is really an intersection between computation embodiment of agents and robustness. Uh possible I think very cool research ideas could be uh integrating deep reinforcement learning. So the the techniques that we the technique that we presented Dr. three is not mutually excluding with deep reinforcement learning. One could use deep reinforcement learning to learn ambiguity. One could use deep reinforcement learning to to learn a better cost etc. So I think there is a sweet spot of applications that can have a high impact um for real world deployments by merging these two um these two approaches. There are also there is also another I'm not a neuroscientist. I'm going to start with that. But uh I think the the model has some cognitive implications as OA just showed uh it supports Bayesian belief updates. But one thing that we had no time to discuss about of course is that something very nice happens in the regimes of large and small ambiguities. So basically when ambiguity is small, you get the behavior of an agent that knows everything about its model. And it turns out that nobody cannot perform this agent. So it's the best possible agent. It is an oracle

essentially that does the best possible actions will achieve the best cost function. On the other side of the spectrum is that when ambiguity is very high there is a sort of a transition where we can show it is shown in the in the preprint that the optimal policy is training independence in the sense that is independent on the training model. So it is a kind of situation where the agent dominated by ambiguity grounds its decisions just on ambiguity and not on what it learned previously which is I think it's a kind of cool behavior. Of course I'm not a neuroscientist but I think it's something that could be investigated uh on on that side on that spectrum on that side of the spectrum. I hope I answered your question. Yeah, that's an interesting point. Kind of like neuroplasticity and age and uh how you never do worse by knowing more about the environment, all things being equal. And then as you turn up the dial on ambiguity, it erases whatever understanding was made ambiguous. Yeah. and and and that could be even empirically fit to understand like what are the biological contexts that different ambiguities different hyperparameters are in play. Yeah. Yeah. Exactly. Yeah. Well, thank you both and to Carl for joining. We really appreciate the work and for for sharing it and having to open source code. It's really cool. Thank you. Uh I put here two links to the papers in and the code uh QR codes in case you want to you are curious and you want to check them out. And thank you again Daniel for organizing this. Awesome. Okay, till next time. Bye bye. Bye.